

Cybersecurity Internship – Week 3 Report

Security Assessment of OWASP Juice Shop Web Application

Student Information

- **Name:** Syed Muhammad Talal
- **Intern ID:** DHC-476
- **Organization:** Developer Hub Corporation

1. Introduction

The aim of this activity was to conduct a basic security assessment for an environment created using the Node.js web application. This involved the detection of web vulnerabilities, adopting secure programming practices, enabling logging in the application, and preparing an assessment report in a professional manner.

2. Objectives

The key objectives of this activity include:

- Conducting basic penetration testing
- Conducting browser testing
- Enabling logging using Winston
- Avoiding insecure programming practices
- Creating a list of cyber security practices
- Capturing vulnerabilities and fixes
- Conducting project submission

3. Tools and Technologies Used

Tool / Technology	Purpose
Node.js	Backend JavaScript runtime
Express.js	Web application framework
bcrypt	Password hashing and salting
Winston	Logging and monitoring
Helmet.js	Security headers implementation

4. Environment Setup

The testing environment included a locally hosted Node.js application running on localhost using Express.js. The project was configured in Visual Studio Code on Windows PowerShell.

The following npm packages were installed:

npm install bcrypt

```
D:\Mega\docs\project\cybersecurity-internship-project>npm install bcrypt

added 3 packages, and audited 85 packages in 7s

23 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

npm install Winston

```
D:\Mega\docs\project\cybersecurity-internship-project>npm install winston

added 25 packages, and audited 110 packages in 6s

24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

npm install helmet

```
D:\Mega\docs\project\cybersecurity-internship-project>npm install helmet

added 1 package, and audited 67 packages in 14s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\Mega\docs\project\cybersecurity-internship-project>node index.js
Server running on http://localhost:3000

D:\Mega\docs\project\cybersecurity-internship-project>
```

5. Basic Penetration Testing

5.1 Nmap Scanning

Nmap was used to scan the local application environment and identify active services and open ports.

Command Used

nmap -sV localhost

Purpose

- Detect running services
- Identify open ports
- Determine service versions

Scan Tools Profile Help

Target: localhost Profile:

Command: nmap -sV localhost

Hosts		Services		Nmap Output	Ports / Hosts	Topology	Host Details	Scans
OS	Host	Port	Protocol	State	Service	Version		
	localhost (127.0.0.1)	135	tcp	open	msrpc	Microsoft Windows RPC		
		445	tcp	open	microsoft-ds			
		902	tcp	open	vmware-auth	VMware Authentication Daemon 1.10 (Uses VNC, SOAP)		
		912	tcp	open	vmware-auth	VMware Authentication Daemon 1.0 (Uses VNC, SOAP)		
		3000	tcp	open	ppp			
		8000	tcp	open	http	Splunkd httpd		
		8089	tcp	open	http	Splunkd httpd		
		8194	tcp	open	http	Golang net/http server		

nmap -T4 -A -v nmap 127.0.0.1

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-14 10:35 -0700
NSE: Warning: Could not load 'mongodb-databases.nse': no path to file/directory: mongodb-databa
NSE: Warning: Could not load 'mongodb-info.nse': no path to file/directory: mongodb-info.nse
NSE: Loaded 156 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 10:35
Completed NSE at 10:35, 0.00s elapsed
Initiating NSE at 10:35
Completed NSE at 10:35, 0.00s elapsed
Initiating NSE at 10:35
Completed NSE at 10:35, 0.00s elapsed
Initiating Parallel DNS resolution of 1 host. at 10:35
Completed Parallel DNS resolution of 1 host. at 10:35, 2.02s elapsed
Initiating System DNS resolution of 1 host. at 10:35
Completed System DNS resolution of 1 host. at 10:35, 2.31s elapsed
Failed to resolve "nmap".
Initiating SYN Stealth Scan at 10:35
Scanning localhost (127.0.0.1) [1000 ports]
Discovered open port 445/tcp on 127.0.0.1
Discovered open port 135/tcp on 127.0.0.1
Discovered open port 3000/tcp on 127.0.0.1
Discovered open port 912/tcp on 127.0.0.1
Discovered open port 8000/tcp on 127.0.0.1
Discovered open port 8194/tcp on 127.0.0.1
Discovered open port 902/tcp on 127.0.0.1
Discovered open port 8089/tcp on 127.0.0.1
Completed SYN Stealth Scan at 10:35, 0.25s elapsed (1000 total ports)
Initiating Service scan at 10:35
Scanning 8 services on localhost (127.0.0.1)
Completed Service scan at 10:36, 32.39s elapsed (8 services on 1 host)
Initiating OS detection (try #1) against localhost (127.0.0.1)
NSE: Script scanning 127.0.0.1.
Initiating NSE at 10:36
```

Observation

The scan identified the HTTP service running on the application port. This confirmed that the web application was accessible and responding properly.

5.2 Cross-Site Scripting (XSS) Testing

Manual browser-based XSS testing was performed using browser developer tools.

Payload Used

```
<script>alert('XSS')</script>
```

Testing Process

The payload was inserted into application input fields to determine whether malicious JavaScript execution was possible.

Observation

The application behavior was monitored to check:

- Whether the script executed
- Whether inputs were sanitized
- Whether user input validation existed

Result

Basic XSS testing was successfully performed for security assessment purposes.

5.3 SQL Injection Testing

Manual SQL injection testing was performed on login-related input fields.

Payload Used

```
admin' OR '1'='1
```

Purpose

To evaluate whether authentication bypass or database query manipulation was possible.

Observation

The application response was analyzed for:

- Authentication bypass
- Database errors
- Improper input handling

Result

The testing process demonstrated awareness of SQL injection testing techniques.

6. Security Logging Implementation

Application logging was implemented using the Winston logging library.

6.1 Winston Logger Configuration

Logger File

```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({
      filename: 'security.log'
    })
  ]
});

module.exports = logger;
```

6.2 Logger Integration

Application Logging

```
logger.info('Application started');
logger.info('Homepage visited');
```

6.3 Logging Output

A dedicated log file named:

security.log

was automatically generated to store security and application activity logs.

Example Log Entries

```
info: Application started
info: Homepage visited
```

7. Security Headers Implementation

Helmet.js was implemented to improve HTTP security headers.

Implementation

```
const helmet = require('helmet');

app.use(helmet());
```

Security Benefits

Helmet.js helped improve:

- Clickjacking protection
- Content Security Policy support
- MIME-type sniffing protection
- Basic browser security hardening

8. Password Security

bcrypt hashing was implemented to secure user passwords before storage.

Example

```
const hashedPassword = await bcrypt.hash(password, 10);
```

Purpose

- Prevent plaintext password storage
- Improve authentication security
- Reduce credential exposure risks

9. Security Checklist

Security Practice	Status
Input Validation	Implemented
Password Hashing	Implemented
Logging Mechanism	Implemented
Security Headers	Implemented
XSS Testing	Tested
SQL Injection Testing	Tested
HTTPS Usage	Recommended

10. Findings and Observations

Finding	Severity	Status
Potential XSS Testing Surface	Medium	Tested
SQL Injection Input Testing	Medium	Tested
Missing HTTPS on Localhost	Low	Expected in local environment
Logging Initially Missing	Low	Fixed using Winston

Recommendations

The following recommendations are made for future improvement:

- Enabling HTTPS in production environments
- Advanced input sanitization
- Centralized logging systems
- Vulnerability scanning
- Checking for vulnerability in dependencies through npm audit
- Implementing authentication rate limiting
- Using session management in a secure way

12. Conclusion

This project effectively illustrated the use of basic techniques of penetration testing, security logging, password protection, and web application hardening using the Node.js framework.

The assessment increased knowledge about vulnerabilities in web applications such as Cross Site Scripting (XSS), SQL injection attacks, along with implementing secure development techniques like password hashing, security headers, and logging user activity.

The project effectively simulated cybersecurity assessment techniques and gained practical experience in web application security testing and documentation.